Python Programming

Part 5 — Object-Oriented Programming (OOP)

Class Notes

Table of Contents

39. Class, Object, Instance, self, __init__

Object-Oriented Programming (OOP) organizes code around real-world "things" — objects — instead of just functions and variables. A **class** is a blueprint or template that defines what properties (data) and behaviors (functions) something should have. An **object** (also called an **instance**) is an actual thing built from that blueprint, with its own real values.

Term	Meaning
Class	The blueprint/template — defines structure, not actual data
Object / Instance	A real, specific thing created from the class, with its own data
Instance variable	Data that belongs to one specific object (different for each object)
Method	A function defined inside a class, describing what objects of that class can do
self	A reference to "this particular object" — used inside methods to access that object's own data
__init__	A special method that runs automatically when a new object is created; used to set up initial values

Applications

Where this is used in Data Science:

- Machine learning models in Scikit-learn/TensorFlow are Python classes — e.g. `model = LinearRegression()` creates an object, and calling `model.fit(X, y)` calls a method on it
- Custom classes are used to represent structured entities like a "Dataset", a "Customer record", or an "Experiment"

EXAMPLE 1 — SIMPLE CLASS WITH __INIT__ AND A METHOD

```
class Student:
    def __init__(self, name, marks):
        self.name = name      # instance variable
        self.marks = marks

    def display(self):
        print(f"{self.name} scored {self.marks} marks")

s1 = Student("Aman", 85)     # s1 is an object/instance
s2 = Student("Riya", 92)     # s2 is a separate object/instance

s1.display()
s2.display()
```

> **Output:**
>
> Aman scored 85 marks Riya scored 92 marks

EXAMPLE 2 — A BIGGER, MORE REALISTIC CLASS

```
class BankAccount:
    def __init__(self, owner, balance=0):
        self.owner = owner
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount
        print(f"Deposited {amount}. New balance: {self.balance}")

    def withdraw(self, amount):
        if amount > self.balance:
            print("Insufficient balance")
        else:
            self.balance -= amount
            print(f"Withdrew {amount}. New balance: {self.balance}")

account = BankAccount("Karan", 1000)
account.deposit(500)
account.withdraw(300)
account.withdraw(5000)
```

Output:

```
Deposited 500. New balance: 1500 Withdrew 300. New balance: 1200 Insufficient balance
```

💼 **Common Interview Question:** What is the difference between a class and an object? Why is self needed in every method?

Note: `self` is how Python knows *which* object's data to work with. Without it, a method wouldn't know if it should use `s1`'s data or `s2`'s data — `self` always refers to the object the method was called on.

40. Inheritance — Single, Multiple, Multilevel, Hierarchical, Hybrid/Diamond

Inheritance lets one class (child/derived class) reuse the properties and methods of another class (parent/base class), without rewriting that code. It models "is-a" relationships — e.g. a Dog "is a" type of Animal.

Type	Structure
Single	One child class inherits from one parent class
Multiple	One child class inherits from more than one parent class at the same time
Multilevel	A chain — Class C inherits from Class B, which inherits from Class A
Hierarchical	Multiple child classes all inherit from the same single parent class
Hybrid / Diamond	A combination of the above (e.g. multiple + multilevel), which can create a "diamond" shape in the class structure

Advantages

- Avoids rewriting the same code across related classes
- Models real-world relationships naturally (a Dog is an Animal)

Limitations

- Deep or hybrid inheritance chains can become confusing to trace
- Diamond inheritance can create ambiguity about which parent's method gets used — Python resolves this using MRO (covered next topic)

Applications

Where this is used in Data Science:

- Custom model classes often inherit from a base model class (e.g. a base "Preprocessor" class, with "TextPreprocessor" and "ImagePreprocessor" as children)
- Framework classes like PyTorch's `nn.Module` are inherited from when building custom neural network layers

EXAMPLE 1 — SINGLE INHERITANCE

```
class Animal:
    def speak(self):
        print("Animal makes a sound")

class Dog(Animal):
    def bark(self):
        print("Dog barks")

d = Dog()
d.speak()   # inherited from Animal
d.bark()
```

Output:

```
Animal makes a sound Dog barks
```

EXAMPLE 2 — MULTILEVEL INHERITANCE

```
class Animal:
    def eat(self):
        print("Animal eats food")

class Mammal(Animal):
    def walk(self):
        print("Mammal walks")

class Dog(Mammal):
    def bark(self):
        print("Dog barks")

d = Dog()
d.eat()    # from Animal
d.walk()   # from Mammal
d.bark()   # from Dog
```

Output:

```
Animal eats food Mammal walks Dog barks
```

EXAMPLE 3 — HIERARCHICAL INHERITANCE

```
class Animal:
    def eat(self):
        print("Animal eats food")

class Dog(Animal):
    def bark(self):
        print("Dog barks")

class Cat(Animal):
    def meow(self):
        print("Cat meows")

d = Dog()
c = Cat()
d.eat(); d.bark()
c.eat(); c.meow()
```

Output:

```
Animal eats food Dog barks Animal eats food Cat meows
```

EXAMPLE 4 — MULTIPLE INHERITANCE (AND A HINT OF THE DIAMOND PROBLEM)

```
class Father:
    def skills(self):
        print("Father: good at gardening")

class Mother:
    def skills(self):
        print("Mother: good at cooking")

class Child(Father, Mother):
    pass

c = Child()
c.skills()   # which one runs? decided by MRO — see next topic
```

Output:

```
Father: good at gardening
```

💼 **Common Interview Question:** What is diamond inheritance and why can it cause ambiguity?

41. super(), MRO, Method Overriding

`super()` is used inside a child class to call a method from its parent class — most commonly used inside `__init__` to run the parent's setup code before adding the child's own. **MRO (Method Resolution Order)** is the specific order Python follows to decide which class's method to use when multiple parent classes define the same method (as in multiple/diamond inheritance). **Method overriding** means a child class defines a method with the same name as one in its parent, replacing the parent's version for that child.

Applications

Where this is used:

- `super().__init__()` is used constantly when building custom classes on top of a library's base class, e.g. a custom PyTorch model calling `super().__init__()` to properly set up the base `nn.Module`
- Method overriding lets a subclass customize specific behavior (e.g. a custom data loader overriding how data is read) while reusing the rest of the parent class

EXAMPLE 1 — SUPER() CALLING THE PARENT'S __INIT__

```
class Animal:
    def __init__(self, name):
        self.name = name
        print(f"{self.name} created as an Animal")

class Dog(Animal):
    def __init__(self, name, breed):
        super().__init__(name)   # calls Animal's __init__
        self.breed = breed
        print(f"{self.name} is a {self.breed}")

d = Dog("Tommy", "Labrador")
```

Output:

```
Tommy created as an Animal Tommy is a Labrador
```

EXAMPLE 2 — METHOD OVERRIDING

```
class Animal:
    def speak(self):
        print("Animal makes a sound")

class Dog(Animal):
    def speak(self):              # overrides Animal's speak()
        print("Dog barks")

class Cat(Animal):
    def speak(self):              # also overrides speak()
        print("Cat meows")

for animal in [Animal(), Dog(), Cat()]:
    animal.speak()
```

> **Output:**
>
> Animal makes a sound Dog barks Cat meows

EXAMPLE 3 — CHECKING MRO IN MULTIPLE INHERITANCE

```
class Father:
    def skills(self):
        print("Father: good at gardening")

class Mother:
    def skills(self):
        print("Mother: good at cooking")

class Child(Father, Mother):
    pass

print(Child.__mro__)
c = Child()
c.skills()   # follows MRO order: Child -> Father -> Mother -> object
```

> **Output:**
>
> (<class 'Child'>, <class 'Father'>, <class 'Mother'>, <class 'object'>) Father: good at gardening

> 💼 **Common Interview Question:** What is MRO, and how does Python decide which parent's method to call in multiple inheritance?

> **Note:** Python uses the **C3 Linearization algorithm** to compute MRO — in simple terms, it checks classes left to right, depth-first, but makes sure a parent is never checked before its own children. You can always view it using `ClassName.__mro__` .

42. Access Modifiers — Public, Protected, Private

Access modifiers control how visible/accessible a class's variables and methods are from outside the class. Python doesn't enforce strict access control like Java or C++ — instead it uses naming conventions that developers are expected to respect.

Type	Naming	Meaning
Public	normal_name	Accessible from anywhere, inside or outside the class (default)
Protected	_single_underscore	Meant for internal use within the class and its subclasses; accessible but "please don't touch this from outside" by convention
Private	__double_underscore	Python performs "name mangling" to make it harder to access directly from outside the class

Advantages

- Protects internal implementation details from being changed accidentally
- Makes the intended usage of a class clearer to other developers

Limitations

- Python's privacy is only convention-based — a determined developer can still access "private" attributes
- Overusing private/protected attributes can make debugging and testing harder

Applications

Where this matters:

- Keeping internal state (like a model's raw training data or trained weights) protected so it's changed only through proper methods, avoiding accidental corruption
- Public methods act as a clean, controlled interface for a class, hiding messy internal details

EXAMPLE 1 — PUBLIC, PROTECTED, AND PRIVATE ATTRIBUTES

```
class Student:
    def __init__(self, name, marks, ssn):
        self.name = name          # public
        self._marks = marks       # protected (convention)
        self.__ssn = ssn          # private (name-mangled)

s = Student("Aman", 85, "1234-5678")
print(s.name)         # works fine
print(s._marks)       # works, but not recommended from outside
print(s.__ssn)        # this line will cause an error
```

Output:

```
Aman 85 AttributeError: 'Student' object has no attribute '__ssn'
```

EXAMPLE 2 — ACCESSING A PRIVATE ATTRIBUTE PROPERLY, USING A METHOD

```
class Student:
    def __init__(self, name, marks):
        self.name = name
        self.__marks = marks       # private

    def get_marks(self):           # controlled access
        return self.__marks

    def set_marks(self, new_marks):
        if 0 <= new_marks <= 100:
            self.__marks = new_marks
        else:
            print("Invalid marks")

s = Student("Riya", 90)
print(s.get_marks())
s.set_marks(150)     # rejected by validation
s.set_marks(95)      # accepted
print(s.get_marks())
```

Output:

```
90 Invalid marks 95
```

> 💼 **Common Interview Question:** Since Python doesn't have true private variables, how is __ssn actually still accessible? (Hint: name mangling — it becomes _Student__ssn internally.)

43. Encapsulation

Encapsulation means bundling data (attributes) and the methods that operate on that data together inside one class, while restricting direct outside access to some of that data. It is achieved in Python using private/protected attributes combined with public "getter" and "setter" methods that control how the data is read or changed.

Advantages

- Protects data from being changed in invalid or unexpected ways
- Lets internal implementation change later without breaking code that uses the class

Limitations

- Adds a bit of extra code (getters/setters) compared to just exposing attributes directly
- Can feel like unnecessary overhead for very simple classes

Applications

Where this is used in Data Science:

- A custom Dataset class encapsulating raw data, exposing only safe, validated methods to access or modify it
- Configuration classes for ML pipelines, where settings are validated through setter methods before being applied

EXAMPLE 1 — SIMPLE ENCAPSULATION WITH A GETTER/SETTER

```
class Temperature:
    def __init__(self, celsius):
        self.__celsius = celsius

    def get_celsius(self):
        return self.__celsius

    def set_celsius(self, value):
        if value < -273.15:
            print("Invalid temperature")
        else:
            self.__celsius = value

t = Temperature(25)
print(t.get_celsius())
t.set_celsius(-300)   # invalid, rejected
t.set_celsius(30)
print(t.get_celsius())
```

Output:

```
25 Invalid temperature 30
```

EXAMPLE 2 — A BIGGER EXAMPLE, ENCAPSULATING A DATASET-LIKE CLASS

```
class Dataset:
    def __init__(self, records):
        self.__records = records   # private, protected from direct edits

    def add_record(self, record):
        if isinstance(record, dict):
            self.__records.append(record)
        else:
            print("Record must be a dictionary")

    def get_all_records(self):
        return self.__records

    def count(self):
        return len(self.__records)

data = Dataset([{"name": "Aman", "marks": 85}])
data.add_record({"name": "Riya", "marks": 92})
data.add_record("invalid entry")   # rejected safely

print(data.get_all_records())
print("Total records:", data.count())
```

Output:

```
Record must be a dictionary [{'name': 'Aman', 'marks': 85}, {'name': 'Riya', 'marks': 92}] Total records: 2
```

💼 **Common Interview Question:** What is the real-world benefit of encapsulation — why not just make every attribute public?

44. Polymorphism and Duck Typing

Polymorphism means "many forms" — the same method name can behave differently depending on which object it's called on. This is usually achieved through method overriding across different classes. **Duck typing** is a Python-specific idea related to polymorphism: Python doesn't check an object's exact type before calling a method on it — if the object has that method (behaves like the expected type), Python just calls it. The saying is: "If it walks like a duck and quacks like a duck, it's treated as a duck."

Applications

Where this is used in Data Science:

- Different model classes (LinearRegression, DecisionTree, etc.) all implement a `.fit()` and `.predict()` method — code that calls `model.predict(X)` works the same regardless of which specific model object is passed in
- Duck typing allows writing flexible functions that work on any object with the right methods, without checking exact types

EXAMPLE 1 — POLYMORPHISM THROUGH METHOD OVERRIDING

```
class Shape:
    def area(self):
        pass

class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius
    def area(self):
        return 3.14 * self.radius ** 2

class Rectangle(Shape):
    def __init__(self, length, width):
        self.length = length
        self.width = width
    def area(self):
        return self.length * self.width

shapes = [Circle(5), Rectangle(4, 6)]
for shape in shapes:
    print(f"{type(shape).__name__} area:", shape.area())
```

> **Output:**
>
> Circle area: 78.5 Rectangle area: 24

EXAMPLE 2 — DUCK TYPING, NO SHARED PARENT CLASS NEEDED

```
class Duck:
    def sound(self):
        print("Quack quack")

class Dog:
    def sound(self):
        print("Woof woof")

class Robot:
    def sound(self):
        print("Beep beep")

def make_it_speak(entity):
    entity.sound()   # works on ANY object with a sound() method

for e in [Duck(), Dog(), Robot()]:
    make_it_speak(e)
```

Output:

```
Quack quack Woof woof Beep beep
```

💼 **Common Interview Question:** What is duck typing, and how is it different from traditional polymorphism through inheritance?

45. @staticmethod, @classmethod, @property

These decorators change how a method behaves in relation to the class and its instances.

Decorator	Meaning
Regular method	Takes `self`, works with a specific object's data
@staticmethod	Does not take `self` or `cls` — behaves like a plain function placed inside the class for organizational purposes, no access to instance or class data
@classmethod	Takes `cls` instead of `self` — works with the class itself, not one specific object; often used for alternative constructors
@property	Lets a method be accessed like an attribute (without parentheses), useful for computed/derived values

Applications

Where this is used:

- @staticmethod: utility/helper functions related to a class, e.g. a validation check that doesn't need any object data
- @classmethod: alternative constructors, e.g. creating a Dataset object directly from a CSV file path
- @property: exposing a calculated value (like BMI from height/weight) as if it were a simple attribute

EXAMPLE 1 — @STATICMETHOD

```
class MathUtils:
    @staticmethod
    def is_even(n):
        return n % 2 == 0

print(MathUtils.is_even(10))
print(MathUtils.is_even(7))
```

> **Output:**
>
> ```
> True False
> ```

EXAMPLE 2 — @CLASSMETHOD AS AN ALTERNATIVE CONSTRUCTOR

```
class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

    @classmethod
    def from_string(cls, data_string):
        name, marks = data_string.split("-")
        return cls(name, int(marks))

s = Student.from_string("Aman-85")
print(s.name, s.marks)
```

> **Output:**
>
> ```
> Aman 85
> ```

EXAMPLE 3 — @PROPERTY FOR A COMPUTED VALUE

```
class Rectangle:
    def __init__(self, length, width):
        self.length = length
        self.width = width

    @property
    def area(self):
        return self.length * self.width

r = Rectangle(5, 4)
print(r.area)   # accessed like an attribute, no parentheses needed
```

Output:

```
20
```

💼 **Common Interview Question:** What is the difference between @staticmethod and @classmethod?

46. if __name__ == "__main__":

Every Python file has a built-in variable called `__name__`. When a file is run directly, Python sets `__name__` to `"__main__"`. But when that same file is imported into another file as a module, `__name__` is set to the file's actual name instead. This check lets you write code that only runs when the file is executed directly — not when it's imported elsewhere.

Applications

Where this is used:

- Adding test code or example usage inside a script, without it running automatically every time the file is imported into a bigger project
- Standard practice in almost every real-world Python script, including data science pipelines and ML training scripts

EXAMPLE 1 — BEHAVIOR WHEN RUN DIRECTLY VS IMPORTED

```
# file: greetings.py

def greet():
    print("Hello from greetings.py")

print("This always runs, file:", __name__)

if __name__ == "__main__":
    greet()
    print("This only runs when the file is executed directly")
```

Output:

```
This always runs, file: __main__ Hello from greetings.py This only runs when the file is executed directly
```

EXAMPLE 2 — SAME FILE, BUT IMPORTED INTO ANOTHER SCRIPT

```
# file: main_script.py
import greetings

print("main_script.py is running")
```

Output:

```
This always runs, file: greetings main_script.py is running
```

Note: Notice in Example 2 that "Hello from greetings.py" does NOT print — because `__name__` was `"greetings"`, not `"__main__"`, so the if-block was skipped.

💼 **Common Interview Question:** Why is if __name__ == "__main__": considered good practice in Python scripts?

47. Importance of __init__.py

An `__init__.py` file is a special file placed inside a folder to tell Python "treat this folder as a package" (a collection of importable modules), rather than just a plain folder. It can be completely empty, or it can contain initialization code that runs automatically whenever the package is imported.

Advantages

- Allows organizing large data science projects into clean, importable sub-packages (e.g. `data/`, `models/`, `utils/`)
- Can be used to control exactly what gets exposed when someone imports the package

Limitations

- Modern Python (3.3+) supports "namespace packages" without __init__.py, but including it is still the clearer, more explicit, and widely-used convention

Applications

Where this is used:

- Structuring a data science project into logical packages, e.g. a `preprocessing` package and a `models` package, each importable cleanly

EXAMPLE 1 — A SIMPLE PACKAGE STRUCTURE

```
my_project/
    __init__.py
    utils/
        __init__.py
        helpers.py

# helpers.py
def clean_text(text):
    return text.strip().lower()

# used from outside as:
from my_project.utils.helpers import clean_text
print(clean_text("  Hello World  "))
```

Output:

```
hello world
```

EXAMPLE 2 — __INIT__.PY USED TO SIMPLIFY IMPORTS

```
# utils/__init__.py
from .helpers import clean_text

# now this works directly, without going into helpers.py:
from my_project.utils import clean_text
print(clean_text("  DATA SCIENCE  "))
```

Output:

```
data science
```

Tip: Without `__init__.py` (in older Python versions, or as a matter of clear convention), Python may not recognize a folder as an importable package, causing import errors.

48. Import Rules for Classes

Classes, like functions, are usually defined in one file (module) and then imported into another file where they are needed. Python offers a few different ways to import a class, each with slightly different effects on how you reference it afterward.

Import style	Example	Usage after import
Import the whole module	import models	models.Student(...)
Import a specific class	from models import Student	Student(...)
Import with an alias	from models import Student as St	St(...)
Import everything (not recommended)	from models import *	All public names become directly usable, but this can cause naming conflicts

Advantages

- Keeps large projects organized by splitting classes across logical files
- Aliased imports keep code short and readable

Limitations

- `from module import *` can silently overwrite existing names, causing hard-to-trace bugs
- Circular imports (two files importing each other) can cause errors and need careful project structuring

EXAMPLE 1 — IMPORTING A SPECIFIC CLASS

```
# file: models.py
class Student:
    def __init__(self, name):
        self.name = name

# file: main.py
from models import Student

s = Student("Aman")
print(s.name)
```

> **Output:**
> Aman

EXAMPLE 2 — IMPORTING WITH AN ALIAS, AND IMPORTING MULTIPLE CLASSES

```
# file: models.py
class Student:
    def __init__(self, name):
        self.name = name

class Teacher:
    def __init__(self, name):
        self.name = name

# file: main.py
from models import Student as St, Teacher

s = St("Riya")
t = Teacher("Mr. Sharma")
print(s.name, "-", t.name)
```

> **Output:**
> Riya - Mr. Sharma

> 💼 **Common Interview Question:** Why is "from module import *" generally discouraged in real projects?

49. Package, Module, Function, Class Hierarchy

Python code is organized in layers, from smallest to largest. Understanding this hierarchy helps in structuring any real project, especially larger data science codebases.

Level	What it is	Example
Function	A single reusable block of code performing one task	def clean_text(text): ...
Class	A blueprint grouping related data and functions (methods) together	class Dataset: ...
Module	A single .py file, which can contain functions, classes, and variables	utils.py
Package	A folder containing multiple related modules, marked with __init__.py	my_project/utils/

Visualizing the hierarchy

```
Package (folder, e.g. "my_ml_project")
 └── Module (a .py file, e.g. "preprocessing.py")
      └── Class (e.g. "TextCleaner")
           └── Method (a function inside the class, e.g. "clean()")
      └── Function (standalone, e.g. "load_data()")
```

Applications

Why this structure matters in Data Science:

- Large ML projects are organized exactly this way: a package for the whole project, modules for each stage (data loading, preprocessing, modeling, evaluation), and classes/functions inside each module
- Understanding this hierarchy makes it much easier to read and navigate real-world open-source libraries like Scikit-learn or Pandas

EXAMPLE 1 — A SMALL REALISTIC PROJECT LAYOUT

```
ml_project/
    __init__.py
    data_loader.py        # module
        def load_csv(path): ...       # function
    preprocessing.py      # module
        class TextCleaner:            # class
            def clean(self, text):    # method
                ...
    model.py               # module
        class Model:                  # class
            def train(self, data):    # method
                ...
            def predict(self, X):     # method
                ...
```

EXAMPLE 2 — USING THE FULL HIERARCHY TOGETHER

```
# preprocessing.py
class TextCleaner:
    def clean(self, text):
        return text.strip().lower()

# data_loader.py
def load_data():
    return ["  Hello ", " WORLD  "]

# main.py
from data_loader import load_data
from preprocessing import TextCleaner

raw_data = load_data()             # using a function
cleaner = TextCleaner()            # creating an object (class)
cleaned = [cleaner.clean(text) for text in raw_data]   # using a method

print(cleaned)
```

Output:

```
['hello', 'world']
```

> **Tip:** This function → class → module → package hierarchy is exactly how professional Python libraries (including every data science library you'll use) are structured internally — recognizing it makes reading unfamiliar codebases much easier.